

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Автоэнкодеры»

Выполнила:
Студентка 4 курса
Группы ИИ-25
Мархель А.Д.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: научиться применять автоэнкодеры для осуществления визуализации данных и их анализа.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент с использованием нейросетевой модели автоэнкодера (с двумя и тремя нейронами в среднем слое);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Реализовать метод t-SNE для визуализации данных (использовать также 2 и 3 компонента), построить соответствующую визуализацию;
4. Применить к данным метод PCA (2 и 3 компонента), реализованный в ЛР №1, сделать выводы;
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет.

Вариант 6

№ варианта	Выборка	Класс
6	Wisconsin Diagnostic Breast Cancer (WDBC)	2-й признак

Ход работы

WDBC (Wisconsin Diagnostic Breast Cancer) — набор данных для диагностики рака молочной железы. Признаки вычислены по оцифрованному изображению тонкоигольной аспирационной биопсии и описывают ядра клеток. Выборка содержит 569 объектов и 32 колонки без заголовка: 1-я — ID пациента, 2-я — диагноз (М — malignant, злокачественная опухоль; В — benign, доброкачественная), 3–32 — 30 вещественных признаков. Для каждого из 10 параметров ядра (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension) приведены среднее значение (mean), стандартная ошибка (se) и худшее значение (worst). Фрагмент исходного файла `wdbc.data`:

ID	Diagnosis	radius mean	texture mean	perimeter mean	area mean	...	fractal dim worst
842302	M	17.99	10.38	122.8	1001	...	0.1189
842517	M	20.57	17.77	132.9	1326	...	0.08902
84300903	M	19.69	21.25	130.0	1203	...	0.08758

Diagnosis	Класс	Объектов
B	Benign (доброкачественная)	357
M	Malignant (злокачественная)	212

Подготовка данных (модуль `common.py`): файл загружается через `pandas.read_csv (header=None)` с присвоением имён колонок; колонка ID удаляется как служебный идентификатор, колонка Diagnosis (2-й признак) выделяется в вектор классов y ($B \rightarrow 0, M \rightarrow 1$) и в преобразованиях не участвует — используется только для раскраски графиков. Пропусков в данных нет, на случай их появления предусмотрена замена медианой столбца. Признаки имеют сильно различающиеся масштабы (`area` — сотни и тысячи, `smoothness` — сотые доли), поэтому они стандартизуются (z-оценка).

Для количественного сравнения методов для каждой проекции считаются две метрики: коэффициент силуэта (`silhouette`, от -1 до 1 , чем больше — тем компактнее классы и дальше друг от друга) и точность логистической регрессии на полученных 2–3 компонентах при 5-кратной кросс-валидации (`accuracy`).

Проект состоит из модулей: `common.py` — загрузка, стандартизация, метрики и построение графиков; `autoencoder.py` — автоэнкодер на PyTorch; `tsne.py` — t-SNE (`sklearn.manifold.TSNE`); `pca_lab1.py` — PCA из ЛР №1 (`numpy.linalg.eig`).

Автоэнкодер

Автоэнкодер реализован как полносвязная сеть из линейных слоёв `torch.nn.Linear` с нелинейной функцией активации после скрытых слоёв. Средний слой (`bottleneck`) и выходной слой линейные; выходы среднего слоя (2 или 3 нейрона) используются как координаты проекции. Обучение — по критерию MSE реконструкции, оптимизатор Adam ($lr = 0.001$), 300 эпох, мини-батчи по 32 объекта, фиксированный `seed = 42` для воспроизводимости.

```
class Autoencoder(nn.Module):
    def __init__(self, n_in, n_latent, hidden, act):
        super().__init__()
        self.encoder = self._stack([n_in, *hidden, n_latent], act)
        self.decoder = self._stack([n_latent, *reversed(hidden), n_in], act)

    @staticmethod
    def _stack(sizes, act):
        layers = []
        for i in range(len(sizes) - 1):
            layers.append(nn.Linear(sizes[i], sizes[i + 1]))
            if act is not None and i < len(sizes) - 2:
                layers.append(act())
        return nn.Sequential(*layers)

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z), z
```

t-SNE

t-SNE выполнен с помощью `sklearn.manifold.TSNE` с инициализацией методом PCA (`init='pca'`) для значений perplexity 20, 30, 40, 50, 60 при 2 и 3 компонентах. Наиболее презентативное значение выбирается по максимуму коэффициента силуэта.

```
tsne = TSNE(n_components=n_components, perplexity=perplexity, init="pca",  
            learning_rate="auto", random_state=SEED)  
Z = tsne.fit_transform(Xs)
```

PCA (ЛР №1)

Для сравнения используется функция `pca_method` из ЛР №1 без изменений: ковариационная матрица → `numpy.linalg.eig` → сортировка собственных векторов по убыванию собственных значений → проекция на первые 2 и 3 главные компоненты.

Результат

1. Автоэнкодер: подбор архитектуры

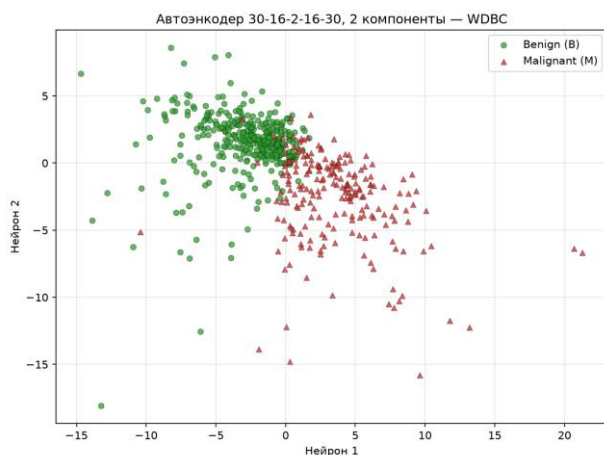
Архитектура подбиралась экспериментально: начиная с простейшей линейной сети и последовательно усложняя её. Для каждой архитектуры обучались сети с 2 и 3 нейронами в среднем слое:

Модель	k	Архитектура	Активация	MSE	Silhouette	Accuracy, %
A	2	30-2-30	линейная	0.3676	0.429	95.08
B	2	30-16-2-16-30	ReLU	0.2741	0.425	94.91
C	2	30-32-16-2-16-32-30	ReLU	0.2429	0.360	94.73
D	2	30-64-32-16-2-16-32-64-30	ReLU	0.2061	0.375	95.26
E	2	30-32-16-2-16-32-30	Tanh	0.2668	0.401	94.55
A	3	30-3-30	линейная	0.2737	0.403	95.08
B	3	30-16-3-16-30	ReLU	0.1983	0.355	95.61
C	3	30-32-16-3-16-32-30	ReLU	0.1716	0.396	94.91
D	3	30-64-32-16-3-16-32-64-30	ReLU	0.1406	0.373	95.26
E	3	30-32-16-3-16-32-30	Tanh	0.1923	0.292	94.38

Выбрана модель B (30-16-k-16-30, ReLU) — самая простая нелинейная архитектура, дающая лучшее среди нелинейных моделей разделение классов: максимальный коэффициент силуэта при $k = 2$ (0.425) и максимальная точность при $k = 3$ (95.61%). Более глубокие сети C и D лучше восстанавливают данные (меньше MSE), но разделение классов не улучшают; активация Tanh (E) уступает ReLU.

Автоэнкодер 30-16-k-16-30 (ReLU):

2 главные компоненты



3 главные компоненты

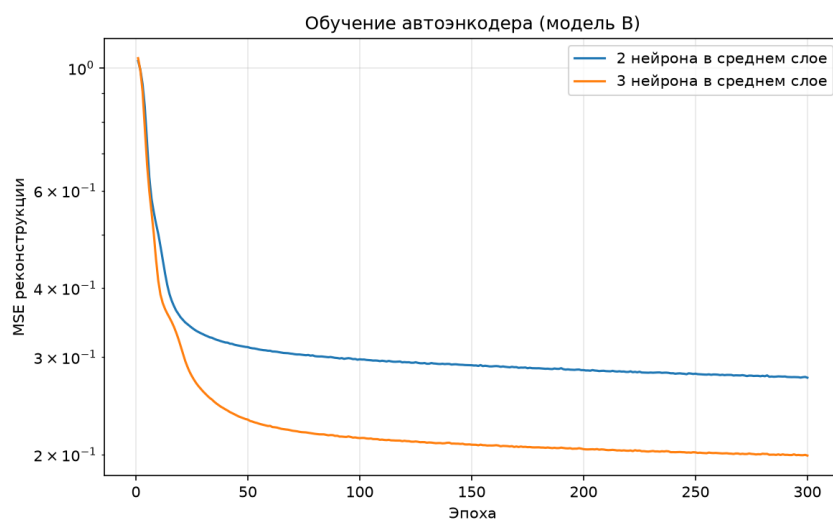
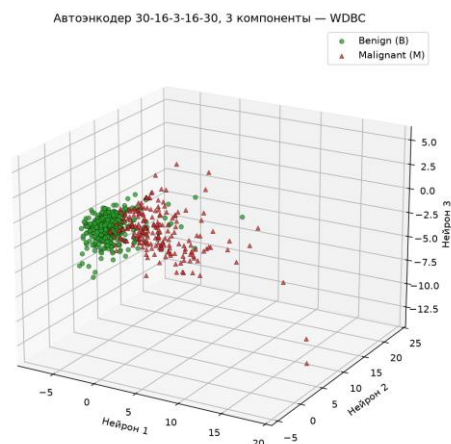


Рисунок — Изменение ошибки реконструкции при обучении автоэнкодера

Загружено объектов: 569, признаков: 30, пропусков заменено: 0
Классы: В = 357, М = 212

Подбор архитектуры (Adam, lr = 0.001, эпох = 300, batch = 32):

Модель	k	Архитектура	Активация	MSE	Silhouette	Accuracy, %
A	2	30-2-30	линейная	0.3676	0.429	95.08
B	2	30-16-2-16-30	ReLU	0.2741	0.425	94.91
C	2	30-32-16-2-16-32-30	ReLU	0.2429	0.360	94.73
D	2	30-64-32-16-2-16-32-64-30	ReLU	0.2061	0.375	95.26
E	2	30-32-16-2-16-32-30	Tanh	0.2668	0.401	94.55
A	3	30-3-30	линейная	0.2737	0.403	95.08
B	3	30-16-3-16-30	ReLU	0.1983	0.355	95.61
C	3	30-32-16-3-16-32-30	ReLU	0.1716	0.396	94.91
D	3	30-64-32-16-3-16-32-64-30	ReLU	0.1406	0.373	95.26
E	3	30-32-16-3-16-32-30	Tanh	0.1923	0.292	94.38

Итоговая модель В: 30-16-2-16-30 / 30-16-3-16-30, активация ReLU

АЕ, 2 нейрона: MSE = 0.2741 | silhouette = 0.425 | accuracy = 94.91%

АЕ, 3 нейрона: MSE = 0.1983 | silhouette = 0.355 | accuracy = 95.61%

2. t-SNE

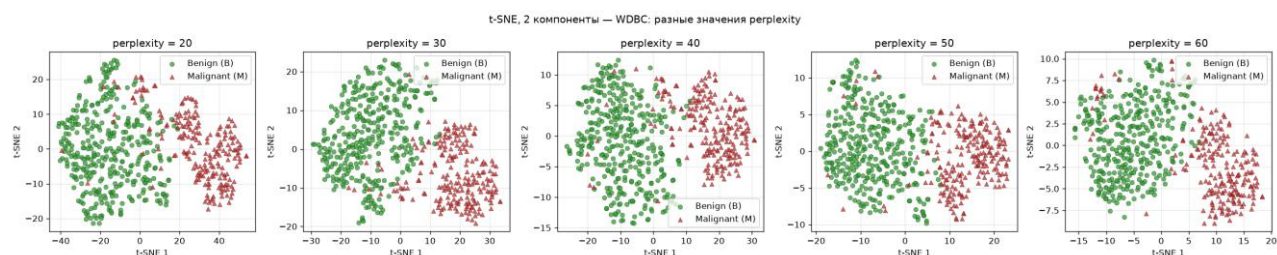


Рисунок — t-SNE, 2 компоненты, perplexity от 20 до 60

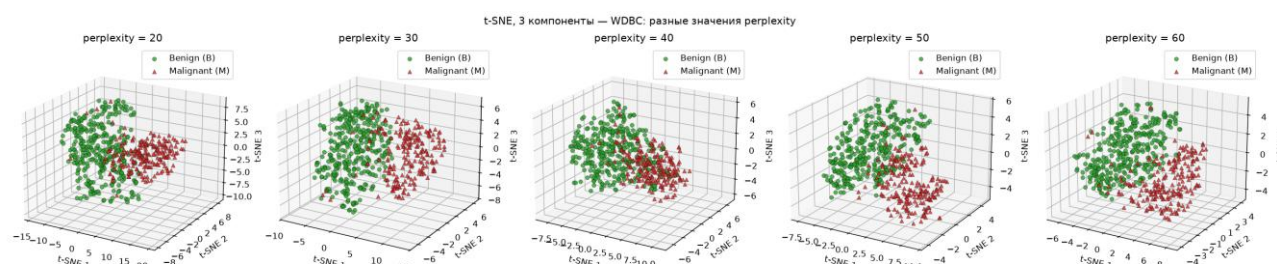


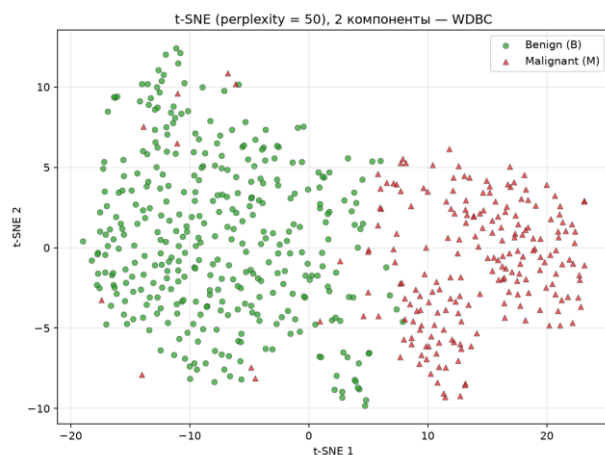
Рисунок — t-SNE, 3 компоненты, perplexity от 20 до 60

k	Perplexity	KL	Silhouette	Accuracy, %
2	20	1.0464	0.481	94.19
2	30	0.9532	0.466	95.95
2	40	0.8623	0.509	95.61
2	50	0.8046	0.515	95.95
2	60	0.7364	0.467	93.49
3	20	0.8612	0.443	95.60
3	30	0.7414	0.410	96.84
3	40	0.6724	0.403	97.37
3	50	0.5829	0.409	96.31
3	60	0.5265	0.411	97.01

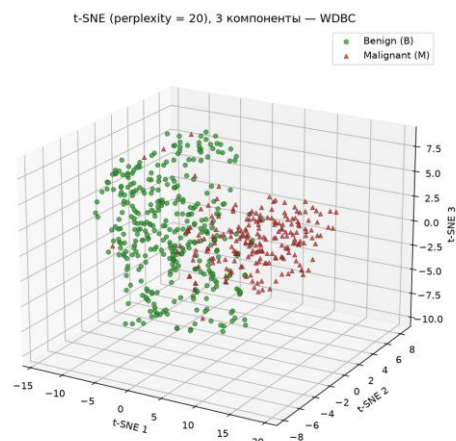
При малой perplexity (20) t-SNE сильнее учитывает локальную структуру и дробит классы на мелкие группы, при большой (60) — сглаживает картину, и граница классов становится менее чёткой. Наиболее презентативными выбраны perplexity = 50 для 2D (silhouette = 0.515, accuracy = 95.95%) и perplexity = 20 для 3D (silhouette = 0.443).

Наиболее презентативные визуализации t-SNE:

2 компоненты (perplexity = 50)



3 компоненты (perplexity = 20)



Загружено объектов: 569, признаков: 30, пропусков заменено: 0
Классы: В = 357, М = 212

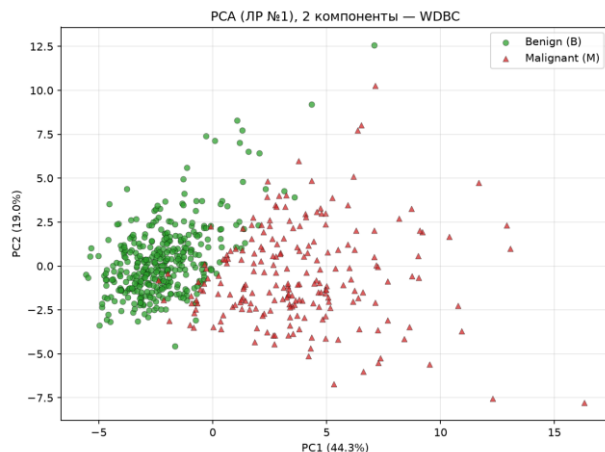
Перебор perplexity (init = 'pca'):

k	Perplexity	KL	Silhouette	Accuracy, %
2	20	1.0464	0.481	94.19
2	30	0.9532	0.466	95.95
2	40	0.8623	0.509	95.61
2	50	0.8046	0.515	95.95
2	60	0.7364	0.467	93.49
3	20	0.8612	0.443	95.60
3	30	0.7414	0.410	96.84
3	40	0.6724	0.403	97.37
3	50	0.5829	0.409	96.31
3	60	0.5265	0.411	97.01

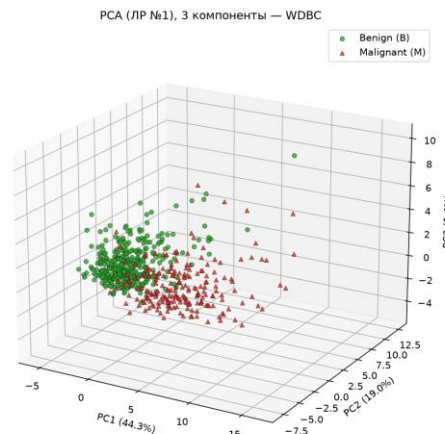
Наиболее презентативная perplexity (максимум silhouette): 2D — 50, 3D — 20

3. PCA (ЛР №1):

2 главные компоненты



3 главные компоненты



Загружено объектов: 569, признаков: 30, пропусков заменено: 0
Классы: В = 357, М = 212

Собственные значения (первые 10 из 30):

λ_1	= 13.3050	(доля дисперсии = 44.27%)
λ_2	= 5.7014	(доля дисперсии = 18.97%)
λ_3	= 2.8229	(доля дисперсии = 9.39%)
λ_4	= 1.9841	(доля дисперсии = 6.60%)
λ_5	= 1.6516	(доля дисперсии = 5.50%)
λ_6	= 1.2095	(доля дисперсии = 4.02%)
λ_7	= 0.6764	(доля дисперсии = 2.25%)
λ_8	= 0.4775	(доля дисперсии = 1.59%)
λ_9	= 0.4176	(доля дисперсии = 1.39%)
λ_{10}	= 0.3513	(доля дисперсии = 1.17%)

Потери информации:

[k=2]	сохранено = 63.24%		потеря = 36.76%
[k=3]	сохранено = 72.64%		потеря = 27.36%

Разделимость классов (silhouette | точность логистической регрессии, 5-fold):

PCA, 2 компоненты:	silhouette = 0.438		accuracy = 95.26%
PCA, 3 компоненты:	silhouette = 0.395		accuracy = 95.08%

Дисперсия WDBC распределена между многими компонентами: первая главная компонента объясняет 44.27%, вторая — 18.97%, третья — 9.39%. Поэтому потери при проецировании велики: 36.76% для 2 компонент и 27.36% для 3 компонент. Тем не менее для разделения классов этого достаточно: точность классификации по 2 компонентам — 95.26%. Наибольшие веса в PC1 имеют признаки вогнутости и размера ядра (concave_points_mean 0.261, concavity_mean 0.258, concave_points_worst 0.251, perimeter_worst 0.237), то есть PC1 — обобщённый показатель «злокачественности» формы ядра; среднее PC1 у класса В равно -2.21, у класса М — 3.71.

Сравнение методов

Метод	k	Silhouette	Accuracy, %	Ошибка восстановления
PCA (JIP №1)	2	0.438	95.26	потеря 36.76%
PCA (JIP №1)	3	0.395	95.08	потеря 27.36%
Автоэнкодер (линейный, A)	2	0.429	95.08	MSE 0.3676
Автоэнкодер (ReLU, B)	2	0.425	94.91	MSE 0.2741
Автоэнкодер (ReLU, B)	3	0.355	95.61	MSE 0.1983
t-SNE (perplexity 50)	2	0.515	95.95	—
t-SNE (perplexity 20)	3	0.443	95.60	—

- Ошибка реконструкции линейного автоэнкодера 30-2-30 ($MSE = 0.3676$) совпала с долей потерянной дисперсии PCA (36.76%), а для 30-3-30 — 0.2737 против 27.36%. Это экспериментально подтверждает, что линейный автоэнкодер с MSE-критерием эквивалентен PCA: он находит то же подпространство главных компонент.
- Нелинейный автоэнкодер (ReLU) сохраняет больше информации, чем PCA: при 2 нейронах ошибка восстановления снижается с 0.368 до 0.274, при 3 — с 0.274 до 0.198. Однако разделимость классов при этом практически не меняется ($\approx 95\%$), так как различие между В и М в WDBC в основном линейное и уже хорошо выражено вдоль первой главной компоненты.
- t-SNE даёт самую наглядную картину: классы образуют два компактных облака с наибольшим коэффициентом силуэта (0.515). Но t-SNE предназначен только для визуализации: расстояния между кластерами и оси не интерпретируются, нет обратного преобразования и способа спроецировать новые объекты.
- Добавление третьей компоненты почти не улучшает разделение ни для одного метода: основная информация о классе содержится в первых двух компонентах.

Вопросы по варианту

1. Как визуализация автоэнкодера (2 и 3 компоненты) отличается от визуализации PCA для данных WDBC — насколько четко разделяются классы?
2. Какая архитектура автоэнкодера (число слоев, нейронов, функция активации) дала наиболее презентативное разделение по 2-му признаку и как это было подобрано экспериментально?

1) Визуализация автоэнкодера в сравнении с PCA

На обоих видах графиков картина качественно одинаковая: доброкачественные опухоли (В) образуют плотное компактное облако, а злокачественные (М) — более разреженное облако, вытянутое в сторону от него. Между классами есть узкая переходная зона, где часть объектов перекрывается; это пограничные случаи, которые не разделяются ни одним из методов.

На графике PCA классы расходятся вдоль первой главной компоненты PC1 (44.3% дисперсии): В слева (среднее -2.21), М справа (3.71), причём облако М имеет почти вдвое больший разброс (СКО 3.02 против 1.64). Оси PCA упорядочены по дисперсии и интерпретируемы. У автоэнкодера оси (выходы нейронов среднего слоя) не упорядочены и не ортогональны, их масштаб произволен, а вид проекции зависит от начальной инициализации весов. В проекции автоэнкодера облако М сильнее «растянуто», и отдельные выбросы уходят далеко от основной массы точек.

Количественно разделение классов у методов близкое: при 2 компонентах silhouette = 0.438 и accuracy = 95.26% у PCA против 0.425 и 94.91% у автоэнкодера; при 3 компонентах accuracy = 95.08% у PCA против 95.61% у автоэнкодера. Нелинейный автоэнкодер лучше восстанавливает исходные данные (MSE 0.274 против 0.368 при 2 нейронах), но это не даёт заметного выигрыша в разделении классов, так как граница между В и М в WDBC близка к линейной. Третья компонента в обоих методах почти не улучшает разделение. Наиболее чёткое визуальное разделение даёт t-SNE (silhouette = 0.515).

2) Подбор архитектуры автоэнкодера

Архитектура подбиралась экспериментально, по рекомендации — от простой к сложной. Все сети обучались одинаково (Adam, lr = 0.001, 300 эпох, batch = 32, MSE, seed = 42) для $k = 2$ и $k = 3$ нейронов в среднем слое. Для каждой сети фиксировались ошибка реконструкции MSE, коэффициент силуэта по 2-му

признаку (диагнозу) и точность логистической регрессии на полученных компонентах (5-кратная кросс-валидация). Были проверены:

- А: 30-k-30, линейная — базовая модель, эквивалентная PCA;
- В: 30-16-k-16-30, ReLU — один скрытый слой в кодере и декодере;
- С: 30-32-16-k-16-32-30, ReLU — два скрытых слоя;
- D: 30-64-32-16-k-16-32-64-30, ReLU — три скрытых слоя;
- Е: 30-32-16-k-16-32-30, Tanh — сравнение функций активации.

Усложнение сети монотонно уменьшает ошибку реконструкции (при $k = 2$: $0.368 \rightarrow 0.274 \rightarrow 0.243 \rightarrow 0.206$), но не улучшает разделимость классов: silhouette у С и D (0.360 и 0.375) ниже, чем у В (0.425). Глубокие сети точнее воспроизводят второстепенные детали данных, не связанные с диагнозом, поэтому классы в их проекции перемешиваются сильнее. Tanh дал худшее разделение, чем ReLU, особенно при $k = 3$ (silhouette 0.292).

Наиболее презентативной выбрана архитектура В: 5 слоёв 30-16-2-16-30 (и 30-16-3-16-30 для 3D), активация ReLU после скрытых слоёв, линейные средний и выходной слои. Это простейшая нелинейная сеть с лучшим балансом: лучший среди нелинейных моделей silhouette при $k = 2$ (0.425) и максимальная точность при $k = 3$ (95.61%), при этом ошибка реконструкции заметно ниже, чем у линейной модели и PCA.

Вывод: научилась применять автоэнкодеры для визуализации и анализа данных. Для выборки WDBC построены проекции на 2 и 3 компоненты с помощью автоэнкодера (PyTorch), t-SNE и PCA из ЛР №1. Линейный автоэнкодер воспроизвёл результат PCA, нелинейный (30-16-k-16-30, ReLU) лучше восстанавливает данные, а t-SNE даёт наиболее наглядное разделение классов. Все методы показывают, что доброкачественные и злокачественные опухоли хорошо различимы ($\approx 95\%$ точности по 2 компонентам), а основная информация о диагнозе сосредоточена в первых двух компонентах.